



Indian Institute of Technology Bombay

Department of Electrical Engineering

Assignment-3

Design, Synthesis, and Verification of 16-bit Kogge-Stone Tree Adder

Subject: VLSI Design

Submitted To:

Dr. Laxmeesha S
Course Instructor
VLSI Design
Course Code: EE671

Submitted By:

GROUP 61
GAURAV KUMAR (25M1159)
ANIKESH JHADI (25M1153)
AMIT KUMAR (25M1165)
JYOTISHMAN DEORI (25M1186)
M.Tech. Electronic Systems (EE5)
Department of Electrical Engineering

October 6, 2025

Contents

1	Introduction	2
1.1	Design Objectives	2
2	Design Methodology	3
2.1	Block Diagram	3
2.2	Module Interface	3
2.3	Design Flow	4
3	Verilog Code and Netlists	5
3.1	Verilog Code for 16-bit Kogge-Stone Adder	5
3.2	Testbench Code	7
3.3	Post-Synthesis RTL Netlist	9
4	Pre-Synthesis Simulation	14
5	Synthesis Results	15
5.1	Synthesis Report	15
5.2	Cell Statistics	18
6	Post-Synthesis Simulation	19
7	Conclusion	21

Chapter 1

Introduction

We have to design and verify a 16-bit **Kogge-Stone Tree Adder (KSA)** with registered inputs and outputs.

The inputs A, B (16 bits each), CIN are clocked into a register using CLK signal. The registered signals are given as input to the adder and the output is again registered using a clock (both SUM and COUT are registered).

1.1 Design Objectives

- Implement a 16-bit Kogge-Stone Tree Adder using Verilog HDL.
- Register the input operands (A, B, and CIN) and output signals (SUM, COUT) with clock and reset.
- Verify the functionality of the design through pre- and post-synthesis simulations.
- Analyze the synthesized reports to extract information such as area and cell count.

Chapter 2

Design Methodology

2.1 Block Diagram

The high-level structure of the registered 16-bit Kogge-Stone Tree Adder is shown in Figure 2.1.

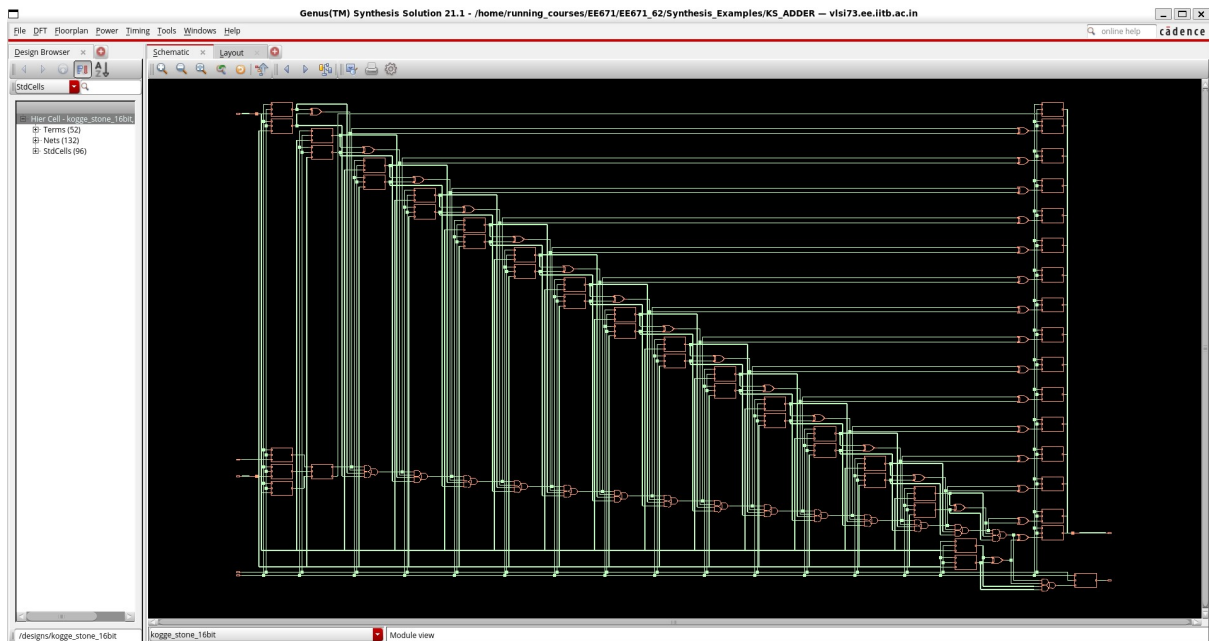


Figure 2.1: Block diagram of 16-bit Kogge-Stone Tree Adder.

2.2 Module Interface

- **Inputs:** A[15:0], B[15:0], CIN, CLK, RST_N
- **Outputs:** SUM[15:0], COUT

2.3 Design Flow

1. RTL design in Verilog.
2. Functional simulation using Incisive.
3. Synthesis in Cadence Genus at 100 MHz.
4. Post-synthesis verification to ensure functional equivalence.

Chapter 3

Verilog Code and Netlists

3.1 Verilog Code for 16-bit Kogge-Stone Adder

```
1  'timescale 1ns/1ps
2  module kogge_stone_16bit (
3      input          CLK,
4      input          RST_N,
5      input  [15:0]  A,
6      input  [15:0]  B,
7      input          CIN,
8      output reg  [15:0] SUM,
9      output reg          COUT
10 );
11
12
13  // Stage 1: Input Registers
14
15  reg [15:0] A_reg, B_reg;
16  reg          Cin_reg;
17
18  always @(posedge CLK or negedge RST_N) begin
19      if (!RST_N) begin
20          A_reg  <= 0;
21          B_reg  <= 0;
22          Cin_reg <= 0;
23      end else begin
24          A_reg  <= A;
25          B_reg  <= B;
26          Cin_reg <= CIN;
27      end
28  end
29
30
31  // Stage 2: Kogge-Stone Generate and Propagate Logic
32
33  wire [15:0] G0, P0;
34  assign G0 = A_reg & B_reg;
35  assign P0 = A_reg ^ B_reg;
```

```

36
37 wire [15:0] G1, P1;
38 wire [15:0] G2, P2;
39 wire [15:0] G3, P3;
40 wire [15:0] G4, P4;
41
42 genvar i;
43
44 // Stage 1 Generation of Propagate and Generate functions
45 generate
46     assign G1[0] = G0[0];
47     assign P1[0] = P0[0];
48     for (i = 1; i < 16; i = i + 1) begin : stage1
49         assign G1[i] = G0[i] | (P0[i] & G0[i-1]);
50         assign P1[i] = P0[i] & P0[i-1];
51     end
52 endgenerate
53
54 // Stage 2 Generation of Propagate and Generate functions
55 generate
56     assign G2[1:0] = G1[1:0];
57     assign P2[1:0] = P1[1:0];
58     for (i = 2; i < 16; i = i + 1) begin : stage2
59         assign G2[i] = G1[i] | (P1[i] & G1[i-2]);
60         assign P2[i] = P1[i] & P1[i-2];
61     end
62 endgenerate
63
64 // Stage 3 Generation of Propagate and Generate functions
65 generate
66     assign G3[3:0] = G2[3:0];
67     assign P3[3:0] = P2[3:0];
68     for (i = 4; i < 16; i = i + 1) begin : stage3
69         assign G3[i] = G2[i] | (P2[i] & G2[i-4]);
70         assign P3[i] = P2[i] & P2[i-4];
71     end
72 endgenerate
73
74 // Stage 4 Generation of Propagate and Generate functions
75
76 generate
77     assign G4[7:0] = G3[7:0];
78     assign P4[7:0] = P3[7:0];
79     for (i = 8; i < 16; i = i + 1) begin : stage4
80         assign G4[i] = G3[i] | (P3[i] & G3[i-8]);
81         assign P4[i] = P3[i] & P3[i-8];
82     end
83 endgenerate
84
85
86 // Carry and Sum Computation

```

```

87
88     wire [16:0] C;
89     assign C[0] = Cin_reg;
90
91     generate
92         for (i = 1; i <= 16; i = i + 1) begin : carry_gen
93             assign C[i] = G4[i-1] | (P4[i-1] & C[0]);
94         end
95     endgenerate
96
97     wire [15:0] SUM_comb;
98     assign SUM_comb = P0 ^ C[15:0];
99     wire COUT_comb = C[16];
100
101
102     // Stage 3: Output Registers
103
104     always @(posedge CLK or negedge RST_N) begin
105         if (!RST_N) begin
106             SUM <= 0;
107             COUT <= 0;
108         end else begin
109             SUM <= SUM_comb;
110             COUT <= COUT_comb;
111         end
112     end
113
114 endmodule

```

Listing 3.1: Verilog Code for 16-bit Kogge-Stone Adder

3.2 Testbench Code

```

1  'timescale 1ns/1ps
2  module tb_kogge_stone_16bit;
3
4      reg    [15:0] A, B;
5      reg          CIN, CLK, RST_N;
6      wire  [15:0] SUM;
7      wire          COUT;
8
9      kogge_stone_16bit uut (
10         .A(A),
11         .B(B),
12         .CIN(CIN),
13         .CLK(CLK),
14         .RST_N(RST_N),
15         .SUM(SUM),
16         .COUT(COUT)
17     );

```

```

18
19 // Clock generation: 100 MHz => 10 ns period
20
21 initial begin
22     CLK = 0;
23     forever #5 CLK = ~CLK;
24 end
25
26 // Corner Test sequence and some general tests
27
28 initial begin
29     $dumpfile("kogge_stone_16bit.vcd");
30     $dumpvars(0, tb_kogge_stone_16bit);
31
32     // Reset
33     RST_N = 0;
34     A = 0; B = 0; CIN = 0;
35     #20;
36     RST_N = 1;
37
38     @(posedge CLK);
39     A = 16'h0000; B = 16'h0000; CIN = 0;
40
41     @(posedge CLK);
42     A = 16'h0001; B = 16'h0001; CIN = 0;
43
44     @(posedge CLK);
45     A = 16'hFFFF; B = 16'h0001; CIN = 0;
46
47     @(posedge CLK);
48     A = 16'h1234; B = 16'h4321; CIN = 0;
49
50     @(posedge CLK);
51     A = 16'hAAAA; B = 16'h5555; CIN = 0;
52
53     @(posedge CLK);
54     A = 16'hFFFF; B = 16'h0001; CIN = 1;
55
56     @(posedge CLK);
57     A = 16'h1234; B = 16'h4321; CIN = 1;
58
59     @(posedge CLK);
60     A = 16'h0FFF; B = 16'h0001; CIN = 1;
61
62     @(posedge CLK);
63     A = 16'h0000; B = 16'h0000; CIN = 1;
64
65     @(posedge CLK);
66     A = 16'hAAAA; B = 16'hAAAA; CIN = 0;
67     #100;
68     $finish;

```

```

69     end
70
71 endmodule

```

Listing 3.2: Testbench for 16-bit Kogge-Stone Adder

3.3 Post-Synthesis RTL Netlist

```

1
2 // Generated by Cadence Genus(TM) Synthesis Solution 21.19-s055_1
3 // Generated on: Oct 5 2025 23:54:28 IST (Oct 5 2025 18:24:28
4 // UTC)
5
6 // Verification Directory fv/kogge_stone_16bit
7
8 module kogge_stone_16bit(CLK, RST_N, A, B, CIN, SUM, COUT);
9     input CLK, RST_N, CIN;
10    input [15:0] A, B;
11    output [15:0] SUM;
12    output COUT;
13    wire CLK, RST_N, CIN;
14    wire [15:0] A, B;
15    wire [15:0] SUM;
16    wire COUT;
17    wire [15:0] B_reg;
18    wire [15:0] A_reg;
19    wire Cin_reg, n_0, n_1, n_2, n_3, n_4, n_5, n_6;
20    wire n_7, n_8, n_9, n_10, n_11, n_12, n_13, n_14;
21    wire n_15, n_16, n_17, n_18, n_19, n_20, n_21, n_22;
22    wire n_23, n_24, n_25, n_26, n_27, n_28, n_29, n_30;
23    wire n_31, n_32, n_33, n_34, n_35, n_36, n_37, n_38;
24    wire n_39, n_40, n_41, n_42, n_43, n_44, n_45, n_46;
25    DFCNQD1 COUT_reg(.CDN (RST_N), .CP (CLK), .D (n_46), .Q (COUT))
26    ;
27    DFCNQD1 \SUM_reg[15] (.CDN (RST_N), .CP (CLK), .D (n_45), .Q
28    (SUM[15]));
29    AO22D0 g3918__2398(.A1 (n_44), .A2 (n_43), .B1 (B_reg[15]), .B2
30    (A_reg[15]), .Z (n_46));
31    XOR2D1 g3919__5107(.A1 (n_44), .A2 (n_43), .Z (n_45));
32    DFCNQD1 \SUM_reg[14] (.CDN (RST_N), .CP (CLK), .D (n_42), .Q
33    (SUM[14]));
34    XOR2D1 g3922__6260(.A1 (n_41), .A2 (n_40), .Z (n_42));
35    AO22D0 g3921__4319(.A1 (n_41), .A2 (n_40), .B1 (B_reg[14]), .B2
36    (A_reg[14]), .Z (n_44));
37    DFCNQD1 \SUM_reg[13] (.CDN (RST_N), .CP (CLK), .D (n_39), .Q
38    (SUM[13]));
39    XOR2D1 g3925__8428(.A1 (n_38), .A2 (n_37), .Z (n_39));
40    AO22D0 g3924__5526(.A1 (n_38), .A2 (n_37), .B1 (B_reg[13]), .B2
41    (A_reg[13]), .Z (n_41));
42    DFCNQD1 \SUM_reg[12] (.CDN (RST_N), .CP (CLK), .D (n_36), .Q

```

```

41         (SUM[12]));
42     AO22D0 g3927__6783(.A1 (n_35), .A2 (n_34), .B1 (B_reg[12]), .B2
43         (A_reg[12]), .Z (n_38));
44     XOR2D1 g3928__3680(.A1 (n_35), .A2 (n_34), .Z (n_36));
45     DFCNQD1 \SUM_reg[11] (.CDN (RST_N), .CP (CLK), .D (n_33), .Q
46         (SUM[11]));
47     AO22D0 g3930__1617(.A1 (n_32), .A2 (n_31), .B1 (B_reg[11]), .B2
48         (A_reg[11]), .Z (n_35));
49     XOR2D1 g3931__2802(.A1 (n_32), .A2 (n_31), .Z (n_33));
50     DFCNQD1 \SUM_reg[10] (.CDN (RST_N), .CP (CLK), .D (n_30), .Q
51         (SUM[10]));
52     AO22D0 g3933__1705(.A1 (n_29), .A2 (n_28), .B1 (B_reg[10]), .B2
53         (A_reg[10]), .Z (n_32));
54     XOR2D1 g3934__5122(.A1 (n_29), .A2 (n_28), .Z (n_30));
55     DFCNQD1 \SUM_reg[9] (.CDN (RST_N), .CP (CLK), .D (n_27), .Q (
56         SUM[9]));
57     AO22D0 g3936__8246(.A1 (n_26), .A2 (n_25), .B1 (B_reg[9]), .B2
58         (A_reg[9]), .Z (n_29));
59     XOR2D1 g3937__7098(.A1 (n_26), .A2 (n_25), .Z (n_27));
60     DFCNQD1 \SUM_reg[8] (.CDN (RST_N), .CP (CLK), .D (n_24), .Q (
61         SUM[8]));
62     AO22D0 g3939__6131(.A1 (n_23), .A2 (n_22), .B1 (B_reg[8]), .B2
63         (A_reg[8]), .Z (n_26));
64     XOR2D1 g3940__1881(.A1 (n_23), .A2 (n_22), .Z (n_24));
65     DFCNQD1 \SUM_reg[7] (.CDN (RST_N), .CP (CLK), .D (n_21), .Q (
66         SUM[7]));
67     AO22D0 g3942__5115(.A1 (n_20), .A2 (n_19), .B1 (B_reg[7]), .B2
68         (A_reg[7]), .Z (n_23));
69     XOR2D1 g3943__7482(.A1 (n_20), .A2 (n_19), .Z (n_21));
70     DFCNQD1 \SUM_reg[6] (.CDN (RST_N), .CP (CLK), .D (n_18), .Q (
71         SUM[6]));
72     AO22D0 g3945__4733(.A1 (n_17), .A2 (n_16), .B1 (B_reg[6]), .B2
73         (A_reg[6]), .Z (n_20));
74     XOR2D1 g3946__6161(.A1 (n_17), .A2 (n_16), .Z (n_18));
75     DFCNQD1 \SUM_reg[5] (.CDN (RST_N), .CP (CLK), .D (n_15), .Q (
76         SUM[5]));
77     AO22D0 g3948__9315(.A1 (n_14), .A2 (n_13), .B1 (B_reg[5]), .B2
78         (A_reg[5]), .Z (n_17));
79     XOR2D1 g3949__9945(.A1 (n_14), .A2 (n_13), .Z (n_15));
80     DFCNQD1 \SUM_reg[4] (.CDN (RST_N), .CP (CLK), .D (n_12), .Q (
81         SUM[4]));
82     AO22D0 g3951__2883(.A1 (n_11), .A2 (n_10), .B1 (B_reg[4]), .B2
83         (A_reg[4]), .Z (n_14));
84     XOR2D1 g3952__2346(.A1 (n_11), .A2 (n_10), .Z (n_12));
85     DFCNQD1 \SUM_reg[3] (.CDN (RST_N), .CP (CLK), .D (n_9), .Q (SUM
86         [3]));
87     AO22D0 g3954__1666(.A1 (n_8), .A2 (n_7), .B1 (B_reg[3]), .B2
88         (A_reg[3]), .Z (n_11));
89     XOR2D1 g3955__7410(.A1 (n_8), .A2 (n_7), .Z (n_9));
90     DFCNQD1 \SUM_reg[2] (.CDN (RST_N), .CP (CLK), .D (n_6), .Q (SUM
91         [2]));

```

```

84  A022D0 g3957__6417(.A1 (n_5), .A2 (n_4), .B1 (B_reg[2]), .B2
85      (A_reg[2]), .Z (n_8));
86  XOR2D1 g3958__5477(.A1 (n_5), .A2 (n_4), .Z (n_6));
87  DFCNQD1 \SUM_reg[1] (.CDN (RST_N), .CP (CLK), .D (n_3), .Q (SUM
      [1]));
88  A022D0 g3960__2398(.A1 (n_2), .A2 (n_1), .B1 (B_reg[1]), .B2
89      (A_reg[1]), .Z (n_5));
90  XOR2D1 g3961__5107(.A1 (n_2), .A2 (n_1), .Z (n_3));
91  DFCNQD1 \SUM_reg[0] (.CDN (RST_N), .CP (CLK), .D (n_0), .Q (SUM
      [0]));
92  FA1D0 g3963__6260(.A (Cin_reg), .B (A_reg[0]), .CI (B_reg[0]),
      .CO
93      (n_2), .S (n_0));
94  CKXOR2D1 g3971__4319(.A1 (B_reg[11]), .A2 (A_reg[11]), .Z (n_31
      ));
95  CKXOR2D1 g3974__8428(.A1 (B_reg[12]), .A2 (A_reg[12]), .Z (n_34
      ));
96  CKXOR2D1 g3975__5526(.A1 (B_reg[2]), .A2 (A_reg[2]), .Z (n_4));
97  CKXOR2D1 g3976__6783(.A1 (B_reg[13]), .A2 (A_reg[13]), .Z (n_37
      ));
98  CKXOR2D1 g3977__3680(.A1 (B_reg[10]), .A2 (A_reg[10]), .Z (n_28
      ));
99  CKXOR2D1 g3978__1617(.A1 (B_reg[9]), .A2 (A_reg[9]), .Z (n_25))
      ;
100 CKXOR2D1 g3973__2802(.A1 (B_reg[15]), .A2 (A_reg[15]), .Z (n_43
      ));
101 CKXOR2D1 g3968__1705(.A1 (B_reg[5]), .A2 (A_reg[5]), .Z (n_13))
      ;
102 CKXOR2D1 g3965__5122(.A1 (B_reg[14]), .A2 (A_reg[14]), .Z (n_40
      ));
103 CKXOR2D1 g3966__8246(.A1 (B_reg[7]), .A2 (A_reg[7]), .Z (n_19))
      ;
104 CKXOR2D1 g3967__7098(.A1 (B_reg[6]), .A2 (A_reg[6]), .Z (n_16))
      ;
105 CKXOR2D1 g3964__6131(.A1 (B_reg[3]), .A2 (A_reg[3]), .Z (n_7));
106 CKXOR2D1 g3969__1881(.A1 (B_reg[4]), .A2 (A_reg[4]), .Z (n_10))
      ;
107 CKXOR2D1 g3970__5115(.A1 (B_reg[8]), .A2 (A_reg[8]), .Z (n_22))
      ;
108 CKXOR2D1 g3972__7482(.A1 (B_reg[1]), .A2 (A_reg[1]), .Z (n_1));
109 DFCNQD1 \A_reg_reg[0] (.CDN (RST_N), .CP (CLK), .D (A[0]), .Q
110      (A_reg[0]));
111 DFCNQD1 \B_reg_reg[10] (.CDN (RST_N), .CP (CLK), .D (B[10]), .Q
112      (B_reg[10]));
113 DFCNQD1 \B_reg_reg[11] (.CDN (RST_N), .CP (CLK), .D (B[11]), .Q
114      (B_reg[11]));
115 DFCNQD1 \B_reg_reg[6] (.CDN (RST_N), .CP (CLK), .D (B[6]), .Q
116      (B_reg[6]));
117 DFCNQD1 \B_reg_reg[12] (.CDN (RST_N), .CP (CLK), .D (B[12]), .Q
118      (B_reg[12]));
119 DFCNQD1 \A_reg_reg[11] (.CDN (RST_N), .CP (CLK), .D (A[11]), .Q

```

```

120         (A_reg[11]));
121 DFCNQD1 \B_reg_reg[4] (.CDN (RST_N), .CP (CLK), .D (B[4]), .Q
122     (B_reg[4]));
123 DFCNQD1 \B_reg_reg[1] (.CDN (RST_N), .CP (CLK), .D (B[1]), .Q
124     (B_reg[1]));
125 DFCNQD1 \B_reg_reg[9] (.CDN (RST_N), .CP (CLK), .D (B[9]), .Q
126     (B_reg[9]));
127 DFCNQD1 \B_reg_reg[8] (.CDN (RST_N), .CP (CLK), .D (B[8]), .Q
128     (B_reg[8]));
129 DFCNQD1 \A_reg_reg[14] (.CDN (RST_N), .CP (CLK), .D (A[14]), .Q
130     (A_reg[14]));
131 DFCNQD1 \A_reg_reg[2] (.CDN (RST_N), .CP (CLK), .D (A[2]), .Q
132     (A_reg[2]));
133 DFCNQD1 \A_reg_reg[4] (.CDN (RST_N), .CP (CLK), .D (A[4]), .Q
134     (A_reg[4]));
135 DFCNQD1 \A_reg_reg[10] (.CDN (RST_N), .CP (CLK), .D (A[10]), .Q
136     (A_reg[10]));
137 DFCNQD1 \B_reg_reg[14] (.CDN (RST_N), .CP (CLK), .D (B[14]), .Q
138     (B_reg[14]));
139 DFCNQD1 \A_reg_reg[5] (.CDN (RST_N), .CP (CLK), .D (A[5]), .Q
140     (A_reg[5]));
141 DFCNQD1 Cin_reg_reg(.CDN (RST_N), .CP (CLK), .D (CIN), .Q (
142     Cin_reg));
143 DFCNQD1 \B_reg_reg[0] (.CDN (RST_N), .CP (CLK), .D (B[0]), .Q
144     (B_reg[0]));
145 DFCNQD1 \A_reg_reg[13] (.CDN (RST_N), .CP (CLK), .D (A[13]), .Q
146     (A_reg[13]));
147 DFCNQD1 \A_reg_reg[9] (.CDN (RST_N), .CP (CLK), .D (A[9]), .Q
148     (A_reg[9]));
149 DFCNQD1 \B_reg_reg[15] (.CDN (RST_N), .CP (CLK), .D (B[15]), .Q
150     (B_reg[15]));
151 DFCNQD1 \B_reg_reg[7] (.CDN (RST_N), .CP (CLK), .D (B[7]), .Q
152     (B_reg[7]));
153 DFCNQD1 \A_reg_reg[15] (.CDN (RST_N), .CP (CLK), .D (A[15]), .Q
154     (A_reg[15]));
155 DFCNQD1 \B_reg_reg[5] (.CDN (RST_N), .CP (CLK), .D (B[5]), .Q
156     (B_reg[5]));
157 DFCNQD1 \A_reg_reg[12] (.CDN (RST_N), .CP (CLK), .D (A[12]), .Q
158     (A_reg[12]));
159 DFCNQD1 \B_reg_reg[13] (.CDN (RST_N), .CP (CLK), .D (B[13]), .Q
160     (B_reg[13]));
161 DFCNQD1 \B_reg_reg[3] (.CDN (RST_N), .CP (CLK), .D (B[3]), .Q
162     (B_reg[3]));
163 DFCNQD1 \A_reg_reg[3] (.CDN (RST_N), .CP (CLK), .D (A[3]), .Q
164     (A_reg[3]));
165 DFCNQD1 \A_reg_reg[6] (.CDN (RST_N), .CP (CLK), .D (A[6]), .Q
166     (A_reg[6]));
167 DFCNQD1 \A_reg_reg[7] (.CDN (RST_N), .CP (CLK), .D (A[7]), .Q
168     (A_reg[7]));
169 DFCNQD1 \A_reg_reg[8] (.CDN (RST_N), .CP (CLK), .D (A[8]), .Q
    (A_reg[8]));

```

```
170   DFCNQD1 \B_reg_reg[2] (.CDN (RST_N), .CP (CLK), .D (B[2]), .Q  
171       (B_reg[2]));  
172   DFCNQD1 \A_reg_reg[1] (.CDN (RST_N), .CP (CLK), .D (A[1]), .Q  
173       (A_reg[1]));  
174 endmodule
```

Listing 3.3: Post-Synthesis Gate-level Netlist (Excerpt)

Chapter 4

Pre-Synthesis Simulation

The pre-synthesis simulation was performed using the iverilog tool, and the waveform obtained is shown in Figure 6.1.

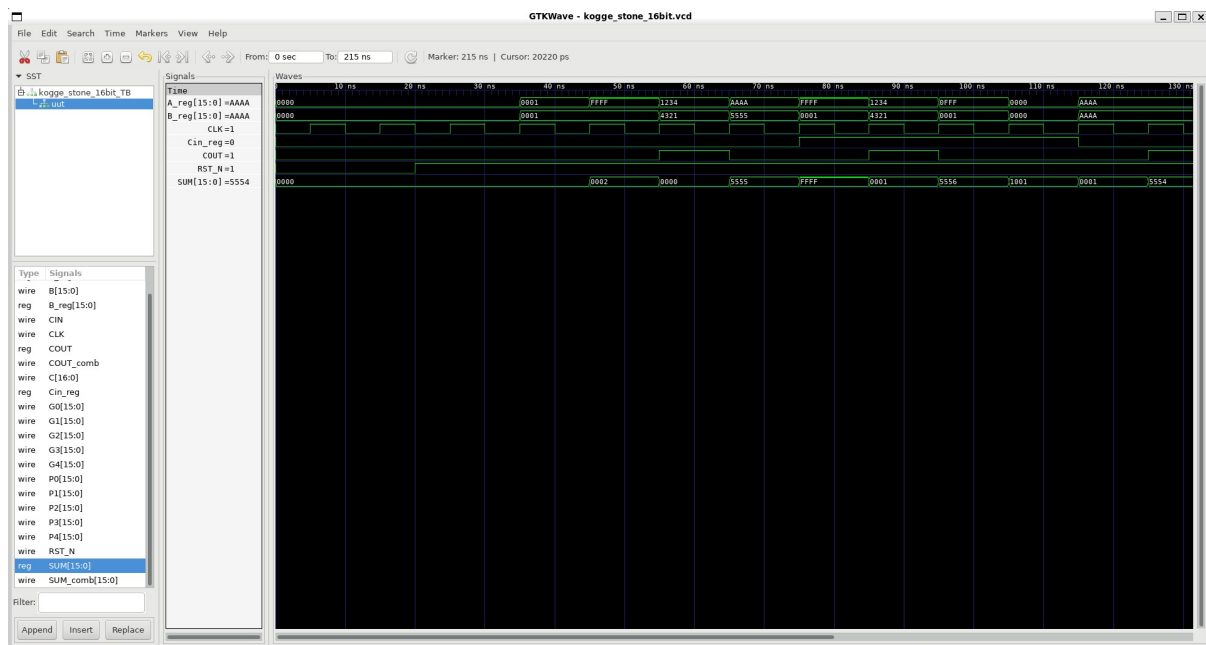


Figure 4.1: Pre-synthesis waveform of the 16-bit Kogge-Stone Adder.

The waveform verifies that the SUM and COUT outputs are functionally correct and synchronized with the clock.

Chapter 5

Synthesis Results

The synthesis was executed in Cadence Genus using a 100 MHz constraint.

5.1 Synthesis Report

Open

summary.rpt
~/Synthesis_Examples/KS_ADDER/Synthesis

Save

=====

Generated by: Genus(TM) Synthesis Solution 21.19-s055_1
Generated on: Oct 05 2025 10:48:27 pm
Module: kogge_stone_16bit
Technology libraries: tcbn65gplusfc_ccs 200
physical_cells
Operating conditions: NCCOM
Interconnect mode: global
Area mode: physical library

=====

Timing

Slack Endpoint Cost Group

+8645ps SUM_reg[15]/D CLK

Area

Instance	Module	Cell Count	Cell Area	Net Area	Total Area
kogge_stone_16bit		96	556.200	154.485	710.685

Design Rule Check

Max_transition design rule: no violations.
Max_capacitance design rule: no violations.

Plain Text ▾ Tab Width: 8 ▾ Ln 1, Col 1 ▾ INS

Figure 5.1: Area and Summary report 1

The image shows a screenshot of a software window titled "EE671_62@vlsi73:~/Synthesis_". The window has a menu bar with "Open", "Save", and a hamburger menu icon. The title bar also includes standard window controls (minimize, maximize, close). The main content area displays the output of a synthesis report. It starts with a header "area.rpt" and a path "~/Synthesis_Examples/KS_ADDER/Synthesis". The report content is as follows:

Generated by: Genus(TM) Synthesis Solution 21.19-s055_1
Generated on: Oct 05 2025 10:48:26 pm
Module: kogge_stone_16bit
Technology libraries: tcbn65gplustc_ccs 200
physical_cells
Operating conditions: NCCOM
Interconnect mode: global
Area mode: physical library

Below this is a table with the following data:

Instance	Module	Cell Count	Cell Area	Net Area	Total Area
kogge_stone_16bit		96	556.200	154.485	710.685

At the bottom of the window, there is a status bar with the text "Plain Text", "Tab Width: 8", "Ln 1, Col 1", and "INS".

Figure 5.2: Area and Summary report 2

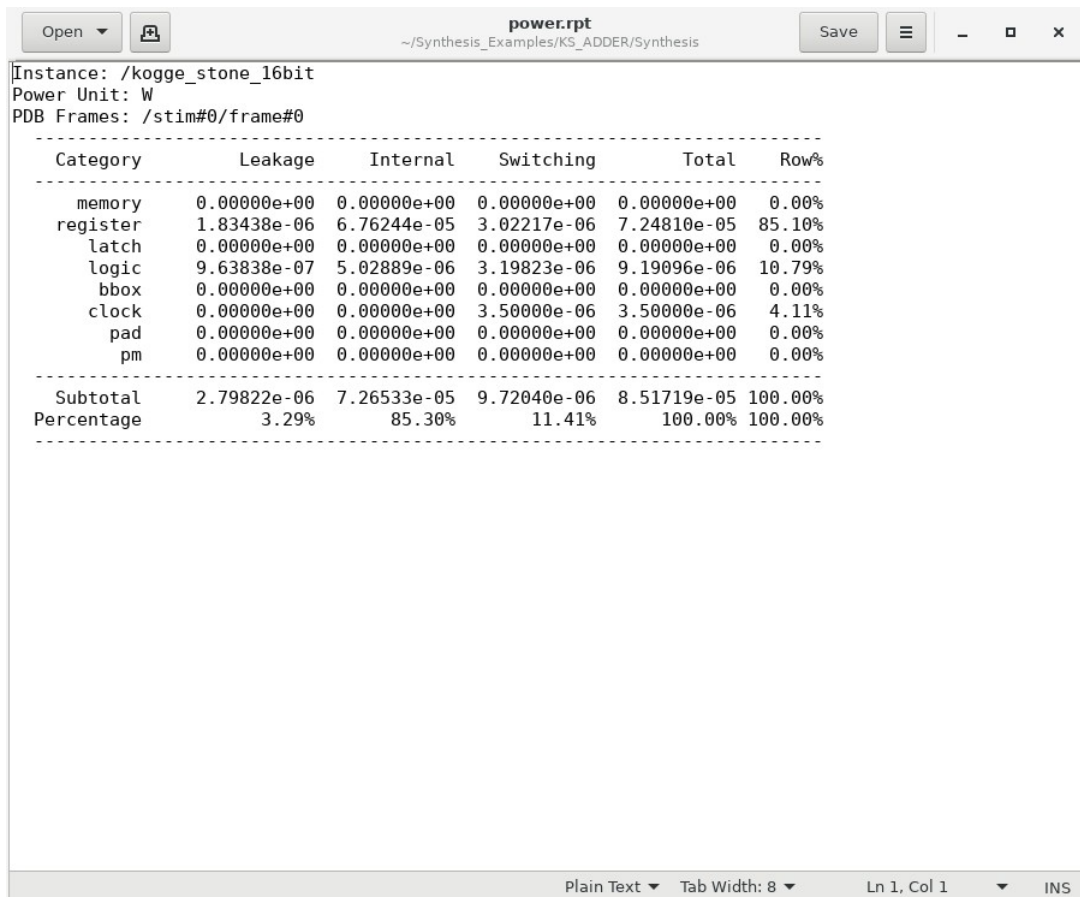


Figure 5.3: Power report

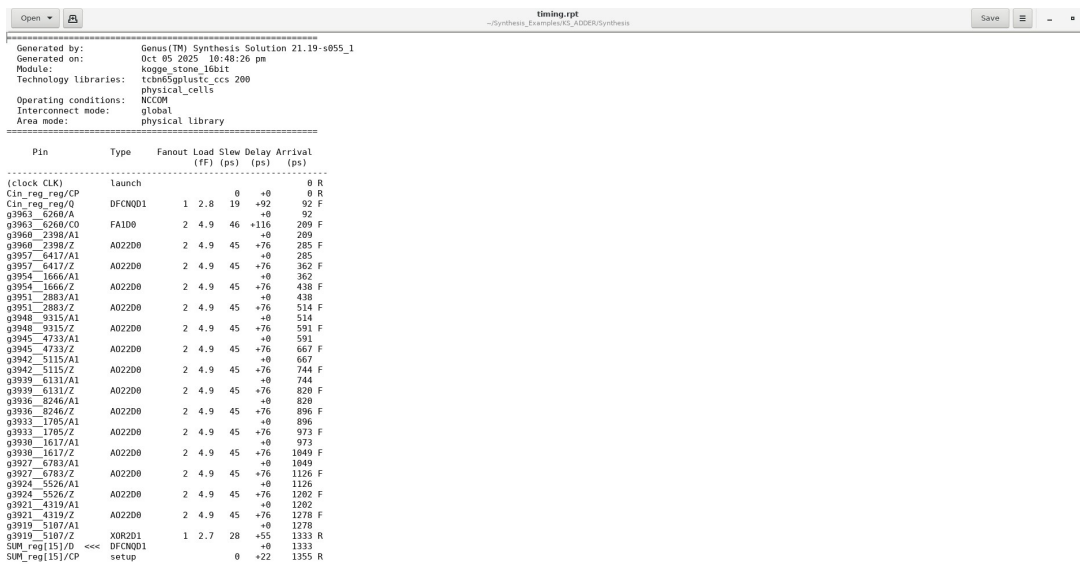


Figure 5.4: Timing report

5.2 Cell Statistics

Open

gates.rpt
~/Synthesis_Examples/KS_ADDER/Synthesis

Save

Generated by:Genus(TM) Synthesis Solution 21.19-s055_1

Generated on:Oct 05 2025 10:48:26 pm

Module:kogge_stone_16bit

Technology libraries:tcbn65gplustc_ccs 200
physical_cells

Operating conditions:NCCOM

Interconnect mode:global

Area mode:physical library

Gate	Instances	Area	Library
A022D0	15	43.200	tcbn65gplustc_ccs
CKX0R2D1	15	54.000	tcbn65gplustc_ccs
DFCNQD1	50	396.000	tcbn65gplustc_ccs
FA1D0	1	9.000	tcbn65gplustc_ccs
X0R2D1	15	54.000	tcbn65gplustc_ccs
total	96	556.200	

Type	Instances	Area	Area %
sequential	50	396.000	71.2
logic	46	160.200	28.8
physical_cells	0	0.000	0.0
total	96	556.200	100.0

Plain Text

Tab Width: 8

Ln 1, Col 61

INS

Figure 5.5: Gates Count report after synthesis.

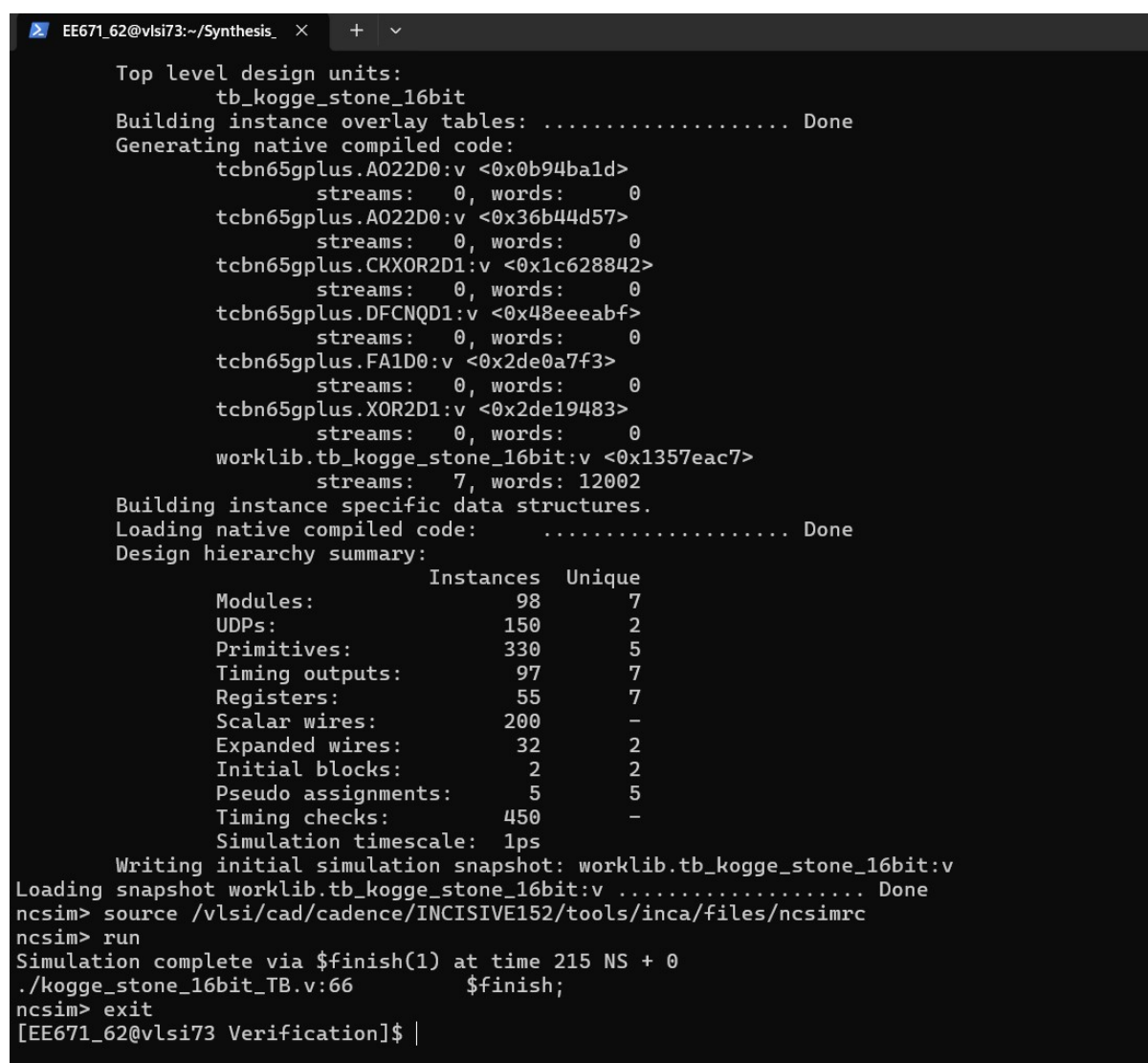
Table 5.1: Cell Statistics from RTL Synthesis Report

Parameter	Count
Number of combinational cells	46
Number of sequential cells (Flip-flops)	50
Total number of cells	96

Chapter 6

Post-Synthesis Simulation

Post-synthesis verification was performed using incisive tool and the same testbench.



```
EE671_62@vlsi73:~/Synthesis_ x + v
Top level design units:
  tb_kogge_stone_16bit
Building instance overlay tables: ..... Done
Generating native compiled code:
  tcbn65gplus.A022D0:v <0x0b94ba1d>
    streams: 0, words: 0
  tcbn65gplus.A022D0:v <0x36b44d57>
    streams: 0, words: 0
  tcbn65gplus.CKXOR2D1:v <0x1c628842>
    streams: 0, words: 0
  tcbn65gplus.DFCNQD1:v <0x48eeeabf>
    streams: 0, words: 0
  tcbn65gplus.FA1D0:v <0x2de0a7f3>
    streams: 0, words: 0
  tcbn65gplus.XOR2D1:v <0x2de19483>
    streams: 0, words: 0
  worklib.tb_kogge_stone_16bit:v <0x1357eac7>
    streams: 7, words: 12002
Building instance specific data structures.
Loading native compiled code: ..... Done
Design hierarchy summary:
      Instances  Unique
Modules:          98      7
UDPs:             150     2
Primitives:       330     5
Timing outputs:   97      7
Registers:        55      7
Scalar wires:     200     -
Expanded wires:   32      2
Initial blocks:   2       2
Pseudo assignments: 5      5
Timing checks:    450     -
Simulation timescale: 1ps
Writing initial simulation snapshot: worklib.tb_kogge_stone_16bit:v
Loading snapshot worklib.tb_kogge_stone_16bit:v ..... Done
ncsim> source /vlsi/cad/cadence/INCISIVE152/tools/inca/files/ncsimrc
ncsim> run
Simulation complete via $finish(1) at time 215 NS + 0
./kogge_stone_16bit_TB.v:66      $finish;
ncsim> exit
[EE671_62@vlsi73 Verification]$ |
```

Figure 6.1: Post-synthesis simulation using incisive

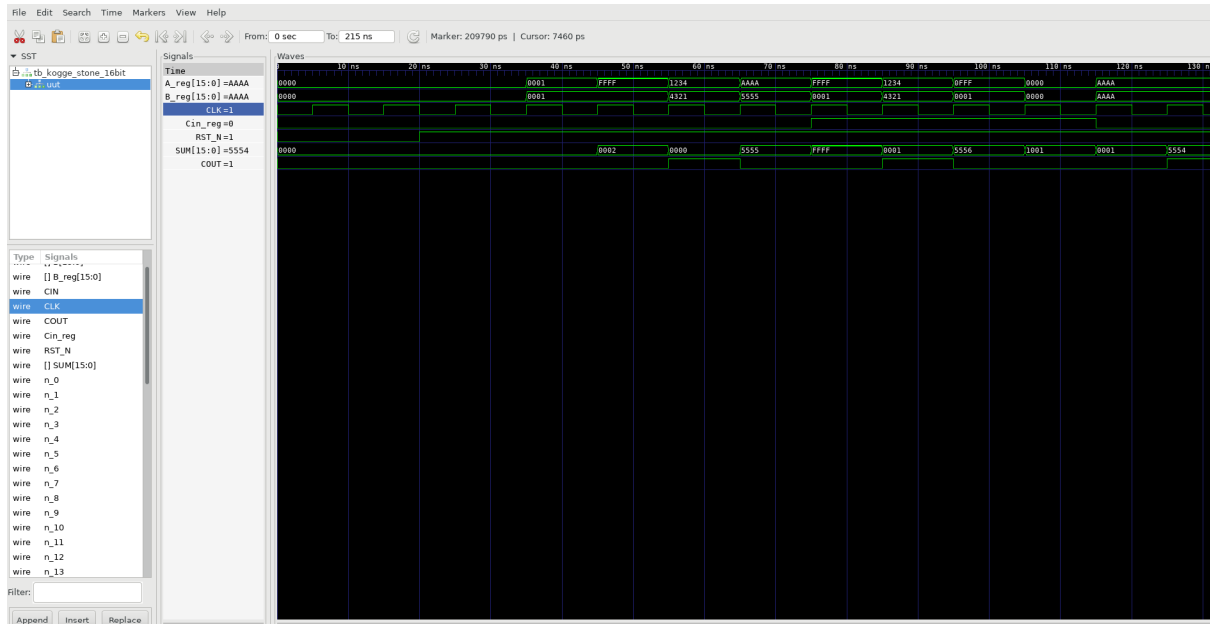


Figure 6.2: Post-synthesis waveform confirming functional equivalence.

Chapter 7

Conclusion

The 16-bit Kogge-Stone Tree Adder was successfully implemented, synthesized, and verified. Both pre- and post-synthesis simulations matched perfectly, confirming the correctness of the RTL design and synthesis flow.

The project demonstrates the high-speed operation of the Kogge-Stone adder architecture and the importance of timing-driven synthesis in achieving reliable performance.